

AI Portfolio – Projektplan

Modulare Streamlit-Anwendung zum Lernen und Vorzeigen von LLM-Kenntnissen. Enno Mühlbauer · Stand September 2026

Grundregeln

- Ein Modul gilt erst als fertig, wenn es **läuft, getestet, dokumentiert und deployed** ist. Erst dann beginnt das nächste.
- Fünf fertige Module sind mehr wert als zehn halbe.
- Logik in `core/`, Oberfläche in `pages/` – strikt getrennt.
- Hartes Ausgabelimit beim API-Anbieter setzen, günstige Modelle für die Demo.
- Nur öffentliche oder selbst erzeugte Daten verwenden, keine dienstlichen Inhalte.

Die sechs Phasen

Phase 1 · Fundament

Infrastruktur

- Repo anlegen, `.env` und `.gitignore` einrichten
- gitleaks als Pre-Commit-Hook aktivieren, Secret Scanning auf GitHub einschalten
- Streamlit-Gerüst mit Passwortabfrage und Navigation bauen
- Leere App direkt deployen (Streamlit Community Cloud)

Fertig, wenn: Die leere App ist online und passwortgeschützt.

Phase 2 · Grundlagen

3 Module

- Gemeinsamer LLM-Client in `core/` (Provider, Retry, Kostenberechnung)
- **Chat** mit Streaming, System-Prompt und Token-/Kostenanzeige
- **Prompt-Labor:** gleiche Aufgabe mit Zero-Shot, Few-Shot, Rolle, Schritt-für-Schritt im Vergleich
- **Datenextraktion:** unstrukturierter Text zu validiertem JSON (Pydantic)

Fertig, wenn: Alle drei Reiter laufen online, Kosten werden angezeigt.

Phase 3 · Kernmodule

3 Module

- **RAG-Chatbot:** PDF-Upload, Chunking, Embeddings, Vektor-DB (Chroma), Antwort mit Quelle und Seitenzahl
- **Tool Use:** Function Calling mit sichtbaren Tool-Aufrufen (z. B. Wetter-API, Rechner)
- **KI-Datenanalyst:** Frage zu SQL, Ausführung mit reinen Leserechten, Tabelle, Diagramm und erzeugter Code

Fertig, wenn: Die App arbeitet mit eigenen Daten und externen APIs.

Phase 4 · Fortgeschritten

3 Module

- **Workflow mit Freigabe:** klassifizieren, Antwort entwerfen, Freigeben/Bearbeiten/Verwerfen, Protokoll
- **MCP-Server:** eigene Tools per Model Context Protocol bereitstellen, Anleitung für Claude Desktop
- **Agent:** Ziel vorgeben, eigene Agenten-Schleife mit Planung, Tool-Aufrufen und Abbruchbedingung

Fertig, wenn: Der Agent löst eine Aufgabe selbstständig und nachvollziehbar.

Phase 5 · Profi-Teil

Qualität und Sicherheit

- **Evaluation:** Testset mit ca. 50 Fragen, LLM-as-a-Judge, Tracing (z. B. Langfuse)
- **Security:** Prompt-Injection-Demo mit Schalter „Schutz an/aus“, PII-Filter, Eingabelimits
- Red-Teaming-Testset mit 20–50 Angriffs-Prompts, Abwehrquote messen (z. B. promptfoo)
- Beides rückwirkend auf die bisherigen Module anwenden

Fertig, wenn: Es gibt echte Messwerte für Antwortqualität und Abwehrquote.

Phase 6 · Feinschliff

Sichtbarkeit

- README mit GIF pro Modul, Architekturskizze, Anleitung zum lokalen Start (Docker)
- SECURITY.md nach OWASP Top 10 for LLM Applications, inkl. ehrlicher Grenzen
- Tests und Sicherheitschecks in GitHub Actions (pytest, pip-audit, Dependabot)
- Abschnitt im Lebenslauf, optional ein LinkedIn-Beitrag

Fertig, wenn: Ein Fremder versteht das Projekt in zwei Minuten.

Projektstruktur

```
app.py          # Startseite, Passwort, Navigation
pages/          # ein File pro Modul (nur Oberfläche)
core/           # Logik: LLM-Client, RAG, Tools, Security
tests/          # pytest
docs/           # dieser Plan, Architekturskizze
eval/           # Testsets und Ergebnisse
```

Technik

Sprache	Python 3.12 (Logik, Oberfläche, Tests)
Oberfläche	Streamlit mit st.navigation, eine Seite pro Modul
Hosting	Streamlit Community Cloud, Secrets für API-Keys, Passwortschutz
Weitere Sprachen	SQL (Datenanalyst), YAML (GitHub Actions), Markdown (Doku)
Bibliotheken	anthropic / openai, pydantic, chromadb, pandas, duckdb, pytest

Weitere Modulideen (optional)

- Mehrere Anbieter plus lokales Modell über Ollama
- Zusammenfassung langer Dokumente per Map-Reduce
- Ticket-Klassifizierung mit Kategorie, Priorität und Zuständigkeit
- Multimodal: Belege per Bild auslesen, Spracheingabe und -ausgabe
- Deployment als API mit FastAPI und Docker, optional Fine-Tuning per LoRA

Was das Projekt beweisen soll

- Praktische Erfahrung mit LLM-APIs, RAG, Tool Use und Agenten
- Bewusstsein für Kosten, Sicherheit und Datenschutz
- Messbare Qualität statt nur funktionierender Demos
- Saubere Architektur, Tests und Dokumentation
- Durchhaltefähigkeit: ein Projekt bis zur Veröffentlichung gebracht

Hinweis: Jede Zeile Code muss im Vorstellungsgespräch erklärbar sein, auch wenn ein KI-Assistent beim Schreiben geholfen hat.